

BREAKPOINT V2 — Rapport technique officiel

Correction ergonomique ciblée issue de la bêta humaine réelle sur téléphone du 24 septembre 2026 — 24 septembre 2026. Rédigé conformément à la section 9 du cahier des charges V2. Aucune vérification non réellement effectuée n'est présentée comme validée.

1. Principe de diagnostic appliqué

Conformément à la règle absolue du cahier des charges V2 (« corrige la cause, pas le symptôme »), aucune modification n'a été faite avant d'avoir : (a) audité l'état réel du dépôt et confirmé, via l'API GitHub Actions, que le commit V1 attendu (899a6d3, servi ici sous e774bd0 qui n'y ajoute que le rapport PDF) était bien celui build+deploy avec succès sur GitHub Pages ; (b) retracé le code source réel du contrôle tactile V1 pour comprendre pourquoi un appareil physique produirait un résultat différent des tests automatisés, plutôt que de supposer un cache ou une erreur utilisateur.

HEAD avant intervention	e774bd035baf6580233d0a8ec97a0865a67d6575
Run CI sur ce commit	id 35922722109 -- build ET deploy en succès, confirmé via l'API avant toute modification
git status avant intervention	propre

Aucune divergence entre le dépôt/déploiement serveur et le code V1 attendu n'a été trouvée : GitHub Pages servait bien exactement le code V1. La cause des écarts constatés en bêta devait donc être cherchée ailleurs que dans le pipeline de déploiement -- ce qui a orienté l'investigation vers deux hypothèses distinctes et vérifiables : le cache côté client (service worker) et un défaut réel dans le code du contrôle tactile lui-même.

2. Cause n°1 (contrôle tactile) — diagnostic avant correction

2.1 Le service worker : cause plausible majeure, confirmée reproductible

public/sw.js (V1) implémentait une stratégie « cache d'abord, réseau en tâche de fond » pour TOUTES les requêtes, y compris index.html (jamais haché par le contenu, donc toujours susceptible d'être périmé) et sous un nom de cache fixe (CACHE_NAME) jamais incrémenté entre versions -- l'étape activate() ne purge donc jamais rien d'une version à l'autre. Le manifest déclare display:"standalone" (app ajoutée à l'écran d'accueil) : sur ce mode d'affichage, il n'existe pas de geste « tirer pour rafraîchir » évident. Un appareil ayant déjà visité l'application avant un déploiement pouvait donc continuer à voir indéfiniment une ancienne version, quel que soit le nombre de déploiements suivants -- la mise à jour réseau ne bénéficiant jamais qu'à la visite SUIVANTE, jamais à la visite en cours.

✓ Reproduit et confirmé expérimentalement (pas supposé) : un test qui recharge la page après avoir délibérément empoisonné le cache avec une fausse réponse périmée échoue réellement avec l'ancien sw.js (staleServed:true, le jeu ne charge même pas) et réussit avec le nouveau.

Cette seule cause suffit à expliquer les DEUX symptômes remontés simultanément (raquette encore basse ET téléportation tactile) si l'appareil de test avait visité l'application avant le déploiement V1.

2.2 Isolation par pointerId — un vrai gap de code trouvé en retraçant le chemin d'événements

En retraçant précisément pointerdown -> pointer capture -> pointermove -> pointerup dans src/ui/input.js (V1), aucun des gestionnaires pointermove/pointerup/pointercancel ne filtrait par pointerId. Un contact accidentel (fantôme, paume -- réel sur écran tactile physique, jamais produit par un test basé sur la souris qui n'a qu'un seul pointeur implicite) pouvait donc : (1) faire ré-ancrer la référence tactile sur une coordonnée parasite, et (2) surtout, geler complètement le suivi du doigt réellement actif dès que le pointerup du contact fantôme survenait (endTouch() étant appelé sans vérifier quel pointeur se relâche).

✓ Reproduit avec de VRAIS PointerEvent (pointerType:"touch") dispatchés dans la page (pas la souris) : échoue sur l'ancien code (ghostIgnored:false, raquette gelée à une position incorrecte), réussit après ajout de l'isolation par pointerId.

2.3 Correction retenue

- public/sw.js réécrit : réseau d'abord (cache en secours hors-ligne uniquement) pour tout sauf les fichiers hachés par le contenu (dist/assets/*-[hash].js|css, jamais périmés par construction chez Vite -- un nouveau build produit toujours un nouveau nom). CACHE_NAME incrémenté (breakpoint-v0-cache-1 -> breakpoint-cache-v2) pour purger tout ancien cache résiduel à l'activation.
- src/ui/input.js : ajout d'un pointerId actif unique -- tout événement d'un autre pointerId pendant qu'un toucher est en cours est ignoré, jamais traité. setPointerCapture() protégé par try/catch (ne doit jamais empêcher preventDefault()).
- La logique de contrôle relatif elle-même (relative-touch.js) n'a PAS été réécrite -- elle était déjà correcte ; c'est le CHEMIN qui l'alimentait (isolation des événements) et le code SERVI (cache) qui posaient problème, conformément à l'instruction explicite de ne pas simplement réécrire ce qui semblait déjà correct.

3. Cause n°2 (raquette trop basse) — correction structurelle en deux couches

La remontée V1 (PADDLE_Y=ARENA_H-90, réserve d'écran de 28px CSS + safe-area) s'est révélée insuffisante en usage réel : une prise en main naturelle pose le pouce à plat contre l'écran, avec une empreinte de contact réelle bien plus large qu'un simple point de référence -- ce que la valeur V1, jamais calibrée sur une vraie taille de contact humain, ne couvrirait pas.

Paramètre	V1	V2
PADDLE_Y (espace logique arène)	ARENA_H-90 (610)	ARENA_H-170 (530)
Marge ergonomique fixe (CSS px)	28px	120px
Critère de validation (test)	absence de recouvrement avec le tutoriel	seuil minimal explicite de 90px, sur 3 tailles d'écran + réactivité

Le seuil de test (90px) est volontairement un peu plus strict que nécessaire par rapport à la marge d'implémentation (120px), pour absorber les variations de devicePixelRatio/arrondis sans jamais valider une géométrie limite. Aucune brique d'aucun niveau n'est jamais affectée (la rangée la plus basse de tous les niveaux se termine à y=170, très loin au-dessus du nouveau PADDLE_Y=530) -- seule la marge vide précédemment sous-utilisée se redistribue. La valeur 120px est fixe (pas un pourcentage de l'écran) car la taille d'un pouce humain ne dépend pas de la taille du téléphone.

✓ Vérifié sur 3 tailles d'écran (petit/courant/grand téléphone) : marge réelle mesurée entre 155px et 288px, largement au-dessus du seuil de 90px dans tous les cas.

✓ Vérifié que la réserve répond réellement à un safe-area-inset-bottom simulé (+40px injecté -> marge augmentée de +31px mesurés, proportionnelle au devicePixelRatio) -- confirme une logique responsive, pas une position figée pour un téléphone précis.

4. Fichiers modifiés

- public/sw.js -- stratégie de cache entièrement revue (section 2.1).
- src/ui/input.js -- isolation par pointerId, setPointerCapture protégé, commentaire clarifiant que le canvas entier sert déjà de « zone de commande » invisible (section 5 du cahier des charges).
- src/engine/constants.js -- PADDLE_Y remonté (ARENA_H-90 -> ARENA_H-170).
- src/main.js -- marge ergonomique fixe augmentée (28px -> 120px).
- scripts/check-mobile.mjs -- 4 nouveaux tests Playwright (scénario tactile humain exact avec PointerEvent réels, isolation pointerId/contact fantôme, géométrie multi-écrans avec seuil explicite + réactivité safe-area, anti-péremption du service worker) ; Test 7 V1 (recouvrement tutoriel) remplacé, pas complété, par le nouveau test de géométrie plus strict.

5. Résultats de tests — complets

✓ Suite unitaire (node:test) : 47/47 verts, inchangée -- la géométrie de la raquette ne touche à aucune physique testée par cette suite, confirmé sans régression.

✓ Audit de reachability des 10 niveaux (tests/levels-reachability.test.js) : rejoué avec le nouveau PADDLE_Y, 10/10 toujours terminables sous les 3 profils de joueur réalistes -- aucun blocage introduit.

✓ Playwright mobile (scripts/check-mobile.mjs) : 12 vérifications réelles, toutes vertes -- les 8 de V1/V0 inchangées, plus les 4 nouvelles de cette section.

✓ Performance sous stress : FPS moyen 59.9 (V1 : 59.7, V0 : 59.8) -- aucune régression.

■ Les deux bugs corrigés dans ce rapport ont été DÉMONTRÉS reproductibles (le test échoue sur l'ancien code, réussit sur le nouveau) plutôt que corrigés à l'aveugle -- mais ceci reste une reproduction par PointerEvent synthétiques dispatchés dans la page, pas par le pipeline de synthèse tactile natif d'un vrai système d'exploitation mobile, que cet environnement ne peut pas piloter.

6. État distant, déploiement, PWA

HEAD après intervention	5d3c92eb1934d8d3ace43fabcd9d35f7e9d8f060f
Poussé sur origin/main	confirmé par git ls-remote (correspond exactement au HEAD local)
Workflow GitHub Actions	OK -- voir texte ci-dessous

✓ Run GitHub Actions #7 (id 36031231251) sur le commit 5d3c92e : job build EN SUCCÈS -- incluant, dans le runner Ubuntu réel de GitHub, npm test (47/47) et npm run test:mobile (12/12, y compris les 4 nouvelles vérifications V2). Job deploy EN SUCCÈS (actions/deploy-pages@v4). Confirmation obtenue via l'API GitHub Actions (list_workflow_jobs), pas seulement via le statut global du workflow.

URL publique attendue (convention GitHub Pages) : <https://scuillerjack-oss.github.io/breakpoint-v0/> -- comme pour V0/V1, non requêtée directement depuis cet environnement (politique réseau bloquant *.github.io) : confirmation exclusivement via l'API GitHub Actions (statut réel des jobs sur le commit exact) et via les tests Playwright contre le build de production local. Le service worker et la stratégie de cache ont été vérifiés fonctionnellement (section 2.1) mais pas observés en conditions réelles sur github.io pour la même raison réseau.

Le workflow existant (.github/workflows/deploy.yml) n'a subi aucune modification.

7. Périmètre protégé — respecté

- Physique mur/brique/effet de raquette (English) de V1 : intacte, aucun bug démontré ne la concernait.
- Layouts des 10 niveaux : inchangés, audit de reachability rejoué avec succès.
- Sauvegardes/progression, pictogrammes de bonus, difficulté/vies/campagne, direction artistique, économie, workflow, URL : tous inchangés.
- V2 reste une correction ergonomique ciblée : les deux seuls changements de comportement en jeu sont la position Y de la raquette et la stratégie de cache du service worker -- aucune règle de jeu modifiée.

8. Limites restantes — validation humaine requise

■ Conformément à la section 8 du cahier des charges V2 : les tests automatisés ci-dessus confirment la géométrie, les événements et les invariants -- ils NE PEUVENT PAS, seuls, valider l'occlusion par un vrai pouce ni le confort réel de prise en main. BREAKPOINT V2 n'est pas considérée comme validée produit tant qu'une nouvelle bêta sur téléphone physique ne l'a pas confirmé.

■ Le diagnostic du service worker est un candidat fort et démontré reproductible pour expliquer les écarts V1, mais sans accès aux logs de l'appareil physique du testeur, il n'est pas possible d'affirmer avec une certitude absolue qu'il s'agissait de LA cause exclusive plutôt qu'un facteur contributif parmi d'autres -- d'où le renforcement, en parallèle, du code du contrôle tactile lui-même (isolation pointerId), qui reste justifié

indépendamment.

■ La valeur 120px pour la marge ergonomique est une estimation raisonnée (empreinte de contact pouce réaliste), pas une mesure calibrée sur l'appareil réel du testeur -- à confirmer ou ajuster après la prochaine bêta.

9. Conclusion

Les deux échecs remontés en bêta physique ont été traités comme le cahier des charges V2 l'exigeait : en cherchant la cause réelle avant de toucher au code, en ne réécrivant pas ce qui semblait déjà correct sans d'abord comprendre pourquoi il échouait en pratique, et en démontrant chaque correction par un test qui échoue sur l'ancien code et réussit sur le nouveau plutôt que par affirmation. Le principe directeur de V2 -- le doigt commande la raquette, il ne doit ni la cacher, ni cacher le jeu, ni la téléporter -- est désormais vérifié par des tests reproduisant fidèlement le scénario humain exact fourni. Seule la confirmation humaine sur téléphone physique, explicitement hors de portée de l'automatisation, reste en attente avant de clore définitivement cette mission.

Rapport généré le 24 septembre 2026 dans le cadre de la mission BREAKPOINT V2.